

RAZORPAY AI REVENUE RECOVERY BUILDATHON

RecoverAI

An AI-Governed Revenue Recovery Decisioning Layer for Razorpay Merchants

"AI proposes. Policy authorizes. Tools execute."

"Detect. Decide. Recover. Verify."

PROJECT DOCUMENTATION

Table of Contents

01	Executive Summary	18	Pending vs. Recovered
02	The Business Problem	19	Outcome Verification (Webhooks)
03	Why the Problem Persists	20	LLM Provenance & Audit Trail
04	Target Users	21	Graceful Degradation
05	Why This Matters — Market Context	22	Frontend / Dashboard Design
06	The Core Insight	23	Technology Stack
07	Proposed Solution	24	Demo Case 1 — Policy Override
08	System Architecture	25	Demo Case 2 — Verified Recovery
09	ML Prediction Layer	26	Batch Experiment & Metrics
10	AI Recovery Agent	27	Key Design Principles
11	Prompt Design & Agent Constraints	28	Different From a Simple Retry Engine
12	AI Output Validation	29	Limitations
13	Deterministic Fallback Logic	30	Future Improvements
14	Policy Engine	31	Buildathon Alignment
15	Why the Policy Engine Is the Core	32	Pitch Narrative
16	Tool Execution Layer	33	Final Mental Model
17	Razorpay Integration		

01 Executive Summary

RecoverAI is an AI-driven **decision and orchestration layer** for revenue recovery, built on top of Razorpay's payment infrastructure. It does not replace existing recovery mechanisms — retries, payment links, and human escalation already exist. Instead, RecoverAI decides **which** mechanism should be applied to **each individual failed payment**, under deterministic financial guardrails, and verifies whether the recovery actually happened.

The system is built around a strict separation of responsibilities:

- **AI (Gemini)** reasons over payment and customer context and proposes one bounded action.
- **A deterministic Policy Engine** authorizes, modifies, or overrides that proposal according to fixed business rules.
- **Tools** execute only the policy-approved action against Razorpay's real APIs.
- **Webhooks** verify whether the action actually resulted in recovered money.
- **An audit trail** records the full decision chain for every case.

This document describes the business problem, the reasoning for using AI at all, the guardrails that make AI use safe in a financial workflow, the technical architecture, two demonstrated end-to-end cases, current limitations, and how the project aligns with the buildathon's evaluation criteria.

02 The Business Problem

It is tempting to describe the problem as *"payments fail and nothing happens."* That framing is inaccurate. Modern payment systems, including Razorpay, already provide recovery mechanisms: automatic retries, payment links, and operational escalation paths.

The real problem shows up at scale. A merchant processing meaningful payment volume can accumulate **thousands of failed payments**, and no two failures are alike:

Dimension	Why it varies
Failure reason	network error, expired card, insufficient funds, invalid payment method, etc.
Transaction amount	₹500 to ₹1,00,000+
Customer history	new customer vs. long-standing customer with high lifetime value
Payment method	card, UPI, netbanking, wallet
Retry attempt number	first failure vs. third consecutive failure
Recovery probability	statistically likely to succeed vs. statistically unlikely
Business/risk constraints	merchant-defined thresholds on autonomous action

Given these dimensions, the same recovery action applied uniformly to every failed payment is a poor strategy. A ₹500 network-error failure from a reliable customer probably just needs a retry. A ₹4,700 expired-card failure needs a payment link, not a retry. A ₹71,000 network-error failure — even with a high recovery probability — may be *too large* for a merchant to want executed autonomously, regardless of how confident the system is.

The actual problem is a decisioning problem:

"Which intervention should be applied to which failed payment, at what point, and under what constraints?"

03 Why the Problem Persists Despite Existing Recovery Mechanisms

Existing recovery mechanisms are **actions**, not **decisioning systems**. A merchant can configure static rules (e.g., "always retry once," or "send a payment link after 2 failed retries"), but static rules cannot easily account for the combinatorial mix of signals described above — amount, failure reason, attempt number, and customer-specific recovery likelihood — without becoming an unmanageable rule tree.

This is precisely the kind of problem where a reasoning system that can weigh multiple contextual signals together is useful — but only if that reasoning stays inside boundaries the merchant controls.

That tension (context-aware reasoning vs. controlled financial execution) is the design problem RecoverAI addresses.

04 Target Users

Primary user — the merchant. Any business processing meaningful volume through Razorpay: e-commerce, SaaS/subscription businesses, EdTech, marketplaces, fintech/BFSI. Their question:

"I have thousands of failed payments. What should I do with each one to maximize recoverable revenue without creating unnecessary risk?"

Secondary / operational users — payments ops, revenue ops, finance, and collections teams. They need visibility into:

- Which payments are at risk
- What RecoverAI recommends
- What action was actually taken
- Why it was taken
- Whether policy changed the AI's recommendation
- Whether money was actually recovered

The payer (customer) is not the primary user of the RecoverAI dashboard — they are the recipient of whichever recovery intervention is chosen (e.g., they receive a payment link).

05 Why This Matters — Market Context

Industry data establishes that failed-payment recovery is a meaningful revenue lever, not that existing recovery is broken:

- Razorpay reports that **more than 50% of customers can drop off** when a payment fails on the first attempt.
- Razorpay reports that **one-click retry can recoup around 15%** of failed payments on the first attempt.
- Stripe has reported meaningful recovery gains from intelligent retry mechanisms.

These figures are used only to support three claims:

1. Failed payments represent meaningful revenue risk.
2. Recovery mechanisms already create measurable value.
3. Therefore, *improving how recovery decisions are made* has direct economic value.

The strongest evidence for RecoverAI's value, however, should come from its own controlled batch experiment (see Section 26), not from third-party industry statistics alone.

06 The Core Insight

Recovery is not one-size-fits-all.

A failed payment should not automatically trigger the same action every time. Effective recovery requires reasoning over context — but financial systems cannot blindly trust an autonomous AI agent with money. RecoverAI resolves this tension by separating the pipeline into five distinct stages:

1. **Prediction** — statistical estimate of recovery likelihood
2. **AI reasoning** — contextual proposal of one action
3. **Deterministic policy** — authorization boundary
4. **Tool execution** — real action against Razorpay
5. **Outcome verification** — proof that money actually moved

This produces the project's central design principle:

"AI proposes. Policy authorizes. Tools execute."

07 Proposed Solution

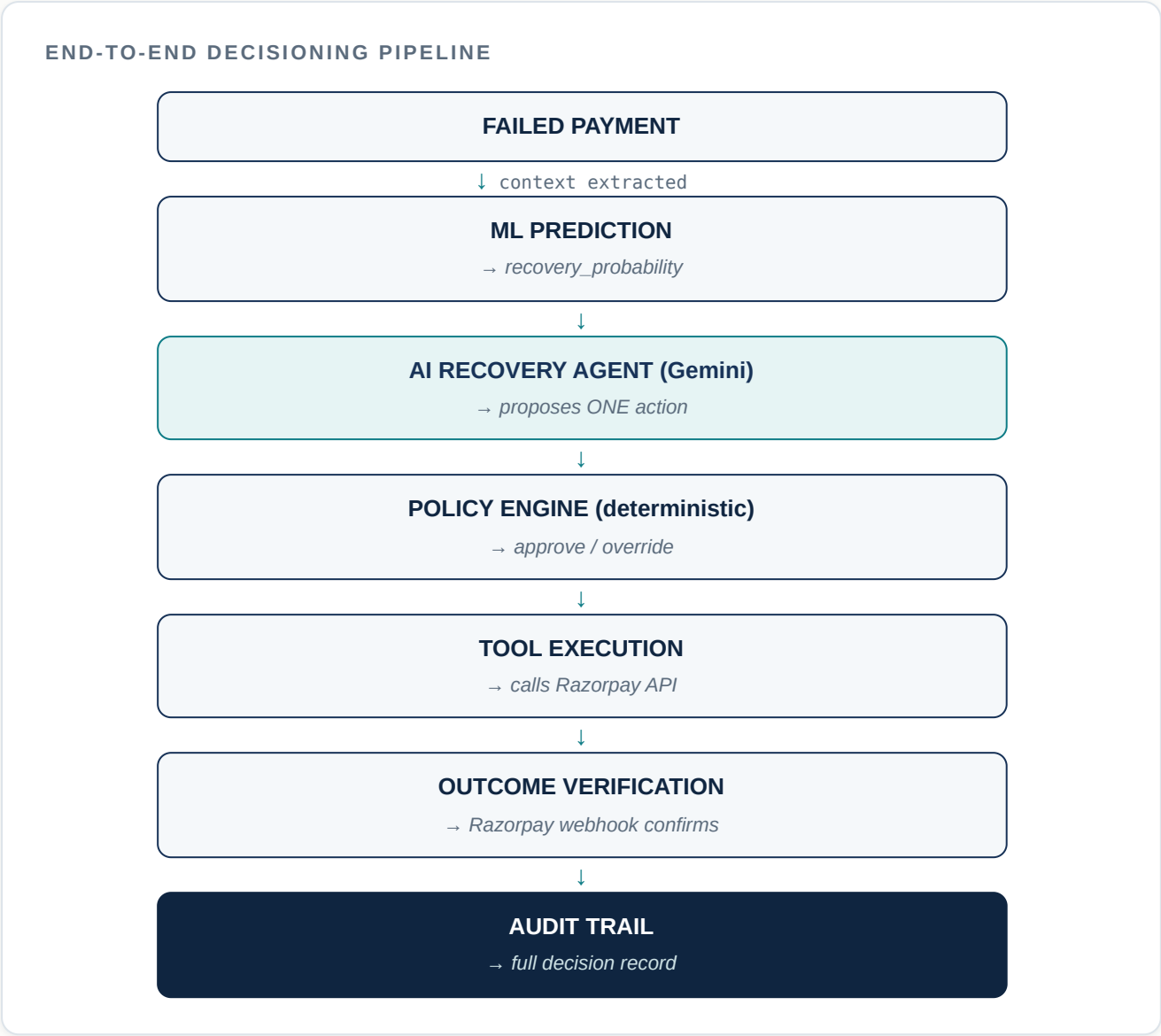
RecoverAI intercepts a failed payment, builds context around it, and runs that context through a governed decision pipeline before any recovery action is taken. Unlike a rules engine, it can weigh

combinations of signals through an LLM-based reasoning step. Unlike an unrestrained AI agent, it cannot execute anything the deterministic Policy Engine hasn't authorized.

The system is not attempting to maximize automation. It is attempting to maximize **safe** automation — recovering as much revenue as possible while keeping every high-risk decision inside merchant-defined boundaries, and while being honest about what has actually been recovered versus what is merely pending.

08 System Architecture

8.1 High-Level Pipeline



8.2 Step-by-Step Flow

1. A payment enters RecoverAI as a failed transaction.
2. The system extracts relevant payment and customer context.
3. The ML layer estimates a recovery probability.
4. The Gemini Recovery Agent reasons about the case and its context.
5. Gemini proposes exactly one bounded action.
6. The Policy Engine evaluates the proposal against deterministic rules.
7. Policy either approves the proposal or overrides it.
8. The approved action is dispatched to the relevant tool.
9. The tool executes the recovery action against Razorpay.
10. The external recovery workflow remains **pending** until an actual payment is confirmed.

11. A Razorpay webhook verifies whether the payment succeeded.
12. The system records the final, verified outcome.
13. A complete decision trail becomes available to operations staff.

09 ML Prediction Layer

The ML layer produces a single output: **recovery probability** — e.g., `recovery_probability = 0.8149`. This number is one *input* into the decision process; it is never treated as the final decision.

Model context / candidate features include:

`transaction_amount``failure_reason``attempt_number``payment_method``customer_success_rate``customer_failure_rate``customer_ltv``avg_payment_amount``days_since_last_success`

Transparency note: some customer-intelligence fields used in the current project are drawn from synthetic/project case data rather than a live Razorpay customer-history API. This document does not claim that every feature is sourced from a production Razorpay data feed.

10 AI Recovery Agent

The Recovery Agent is powered by **Gemini** (current configuration: `gemini-3.6-flash`). It receives the failed-payment case together with its ML-derived recovery probability and customer context, and must propose **exactly one** bounded action from a fixed set:

Action	Meaning
<code>retry</code>	Attempt payment again immediately
<code>schedule_retry</code>	Attempt payment again after a bounded delay (≤ 168 hours)
<code>send_payment_link</code>	Generate a Razorpay Payment Link for the customer
<code>escalate</code>	Route the case to human review
<code>stop</code>	Cease further automated recovery attempts

Along with the action, the agent returns a `confidence` score and a concise `reason` string, both of which are surfaced in the audit trail and dashboard.

11 Prompt Design & Agent Constraints

The agent is explicitly constrained to reduce hallucination and keep its output inside safe bounds. It must:

- Reason **only** from supplied case information — no invented facts.
- Respect the fixed action vocabulary (no free-form actions).
- Never schedule a retry beyond **168 hours**.

- Prefer `send_payment_link` when the payment method is expired or invalid.
- Never attempt to bypass policy constraints (the agent has no execution authority regardless).
- Treat all monetary amounts as **INR**.

The system prompt explicitly states:

"All monetary amounts are in INR (Indian Rupees). Never describe an INR amount as USD, dollars, or any other currency."

This last constraint exists because LLMs trained predominantly on USD-denominated financial text can default to dollar framing unless explicitly corrected — a small but important correctness detail in a financial workflow.

12 AI Output Validation

Gemini's output is not trusted as-is. The RecoveryAgent structurally validates the returned decision — checking that the action is one of the allowed values, that required fields are present, and that the response is well-formed — before it is ever passed to the Policy Engine.

If validation fails, or the LLM call itself fails (timeout, error, malformed response), the system does **not** crash or block the pipeline. It falls back to deterministic logic.

13 Deterministic Fallback Logic

The fallback rules exist purely for **graceful degradation** when the LLM is unavailable or returns an invalid response — they are not the system's primary intelligence layer.

Condition	Sub-condition	Action
attempt_number ≥ 3	—	stop
failure_reason ∈ {expired_card, payment_method_invalid}	probability ≥ 0.45	send_payment_link
	else	stop
failure_reason ∈ {network_error, insufficient_funds}	probability ≥ 0.70	schedule_retry
	probability ≥ 0.50	retry
	else	send_payment_link
otherwise	probability ≥ 0.70	retry
	probability ≥ 0.50	send_payment_link
	else	escalate

14 Policy Engine

The Policy Engine is fully **deterministic** — no LLM involvement — and acts as the safety boundary between AI reasoning and real financial execution.

Current configuration

Parameter	Value
MAX_AUTO_AMOUNT	₹50,000
MIN_AUTO_PROBABILITY	0.30
MAX_RETRY_ATTEMPTS	3

Rules enforced

- 1. Amount > ₹50,000 → **escalate to human**, regardless of AI proposal.
- 2. Attempt number ≥ 3 → **stop**.
- 3. Recovery probability < 0.30 → **stop**.
- 4. Invalid or expired payment method → **block** `retry` / `schedule_retry`; **force** payment-link recovery.
- 5. Only actions from the fixed allowed set may ever be executed.
- 6. Escalation rules cannot be bypassed by the AI under any circumstance.

15 Why the Policy Engine Is the Core of the Project

The Policy Engine is what distinguishes RecoverAI from "send payment data to an LLM and execute whatever it says." The architecture deliberately separates:

AI = reasoning

Policy = authorization

Tools = execution

Example: Policy Override

TXN

Transaction: ₹71,853.45

GEMINI

Proposes `schedule_retry` — confidence: high

POLICY

Checks: amount > ₹50,000 → **OVERRIDE**

FINAL

Action: **escalate to human review**

Reason: "Transaction exceeds ₹50,000 automated recovery threshold"

This is not a case of the AI being "wrong." The AI's proposal can be entirely reasonable given the case context — the override reflects a **stronger business constraint**, not an AI error. The correct framing:

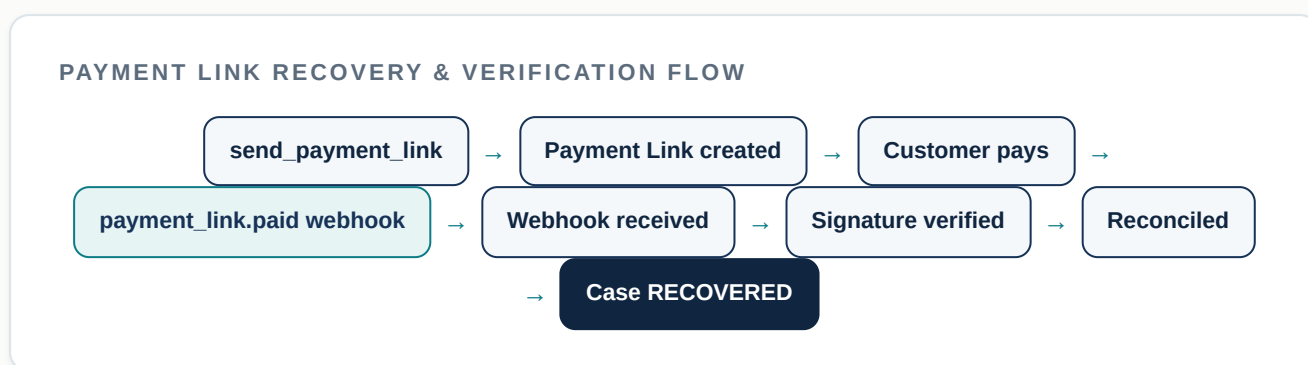
"The AI recommendation was reasonable, but unauthorized under the merchant's policy."

16 Tool Execution Layer

Once the Policy Engine approves an action, a corresponding tool executes it. Tools map 1:1 to the allowed action vocabulary (e.g., a `send_payment_link` tool that calls the Razorpay Payment Links API, a `schedule_retry` tool that queues a delayed retry). The tool layer only ever receives policy-approved actions — it has no visibility into, or ability to act on, the raw AI proposal.

17 Razorpay Integration

RecoverAI integrates with **Razorpay Test Mode** and can create real Razorpay Payment Links as part of the `send_payment_link` action.



The integration uses the Razorpay API, Razorpay Payment Links, Razorpay webhooks, and webhook signature verification. Secret keys and credentials are never exposed in documentation, screenshots, or pitch material.

18 Pending vs. Recovered — A Core Product Principle

Creating a Payment Link does not mean money was recovered.

Immediately after a recovery action is dispatched, the case status is:

OUTCOME_PENDING

Only after Razorpay confirms actual payment via webhook does the status change to:

RECOVERED

This distinction keeps the system financially honest: RecoverAI never claims revenue recovery merely because it *initiated* a recovery action. Revenue is only counted as recovered once it is independently verified by Razorpay.

19 Outcome Verification (Webhooks)

Outcome verification closes the loop between "an action was taken" and "money was actually recovered." Razorpay webhook events (e.g., `payment_link.paid`) are received, their signatures verified, and the underlying payment reconciled against the original failed-payment case before the case is marked `RECOVERED`. Without this step, any recovery-action count could be mistaken for a recovery-revenue count — which the project explicitly avoids conflating.

20 LLM Provenance & Audit Trail

Every decision is auditable. The audit system records, for each case:

agent source (llm / fallback)

provider

model

action proposed

confidence

reason

Recording provenance (`llm` vs. `fallback`) matters because it lets operations staff verify whether a given decision came from actual LLM reasoning or from the deterministic degradation path — a meaningful distinction in a financial audit context.

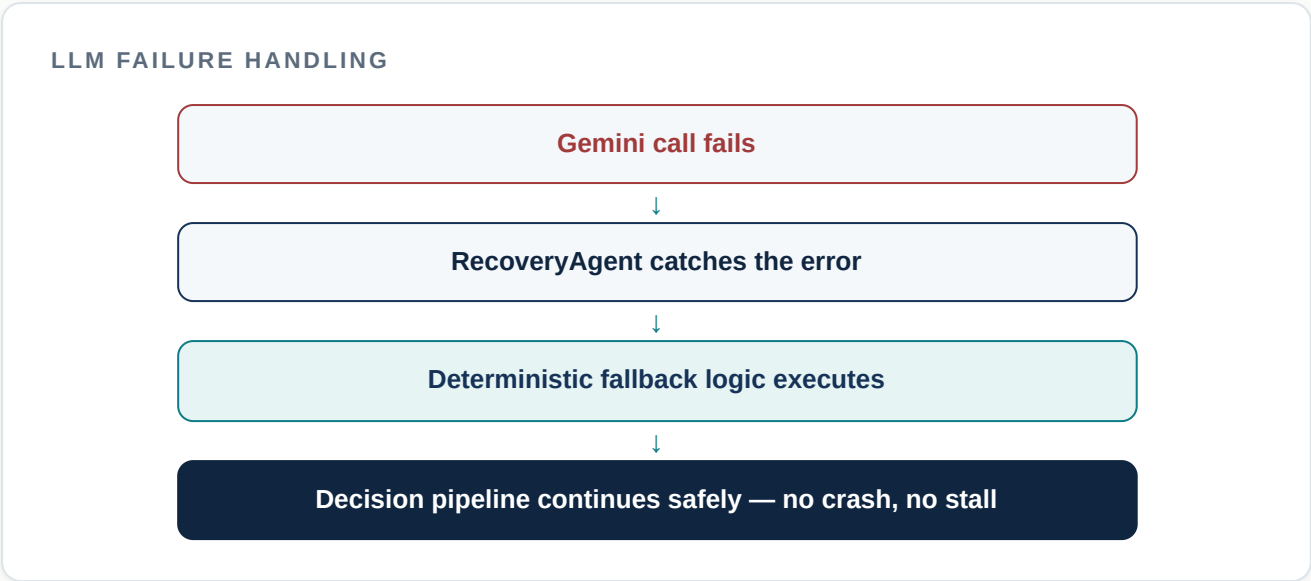


The audit trail should be able to answer, for any case:

- What payment was being recovered?

- What did the ML layer predict?
- What did Gemini recommend, and with what model?
- What did policy decide, and was the AI proposal overridden?
- What tool was executed?
- Was money actually recovered?
- What was the final outcome?
- Did the system use LLM reasoning or fallback logic?

21 Graceful Degradation — LLM Failure Handling

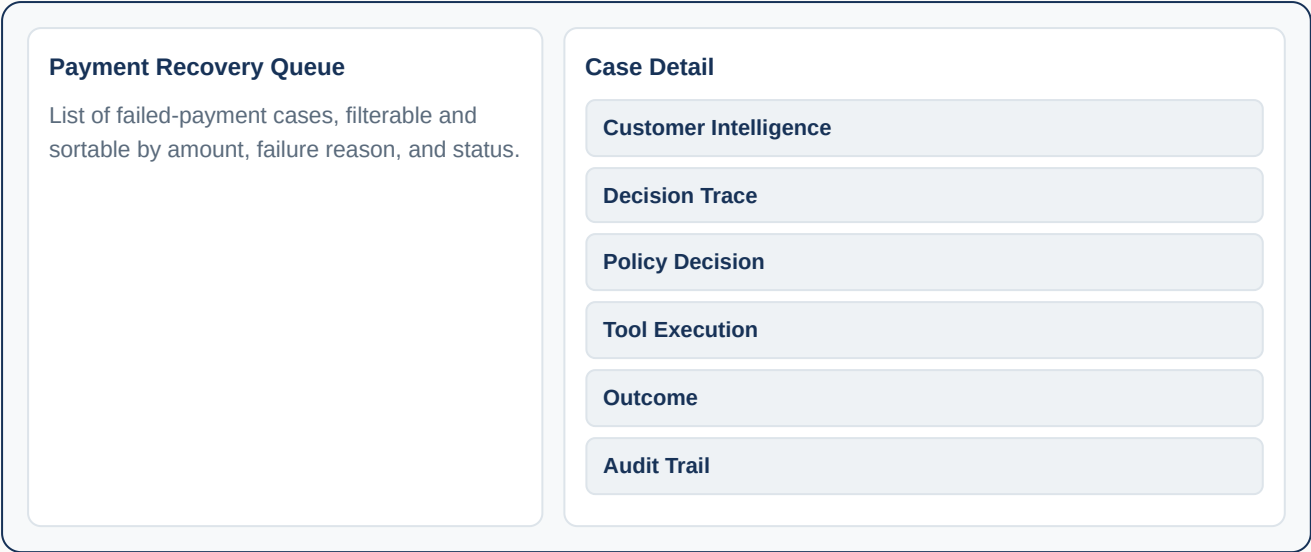


The governing narrative here: **AI availability should never become a single point of failure** in a financial recovery workflow. If Gemini is unreachable or returns an invalid response, the case still receives a valid, policy-compliant decision via the deterministic fallback rules described in Section 13.

22 Frontend / Dashboard Design

The dashboard is built as an **operations interface**, not a consumer-facing product.

Main layout



Customer Intelligence panel shows: success rate, total payments, customer lifetime value, average payment, last successful payment, and failure rate.

Decision Trace (generic template)



Audit trail UI behavior: the latest recovery run for a case is shown expanded and prominent; previous runs are collapsed by default. This keeps operations staff focused on the current decision while preserving full historical evidence on demand, rather than overwhelming them with every past event.

23 Technology Stack

BACKEND

- Python, FastAPI
- Agent layer (Gemini integration)
- ML prediction layer
- Policy Engine
- Tool execution layer
- Razorpay APIs and Webhooks

FRONTEND

- React, Vite, CSS

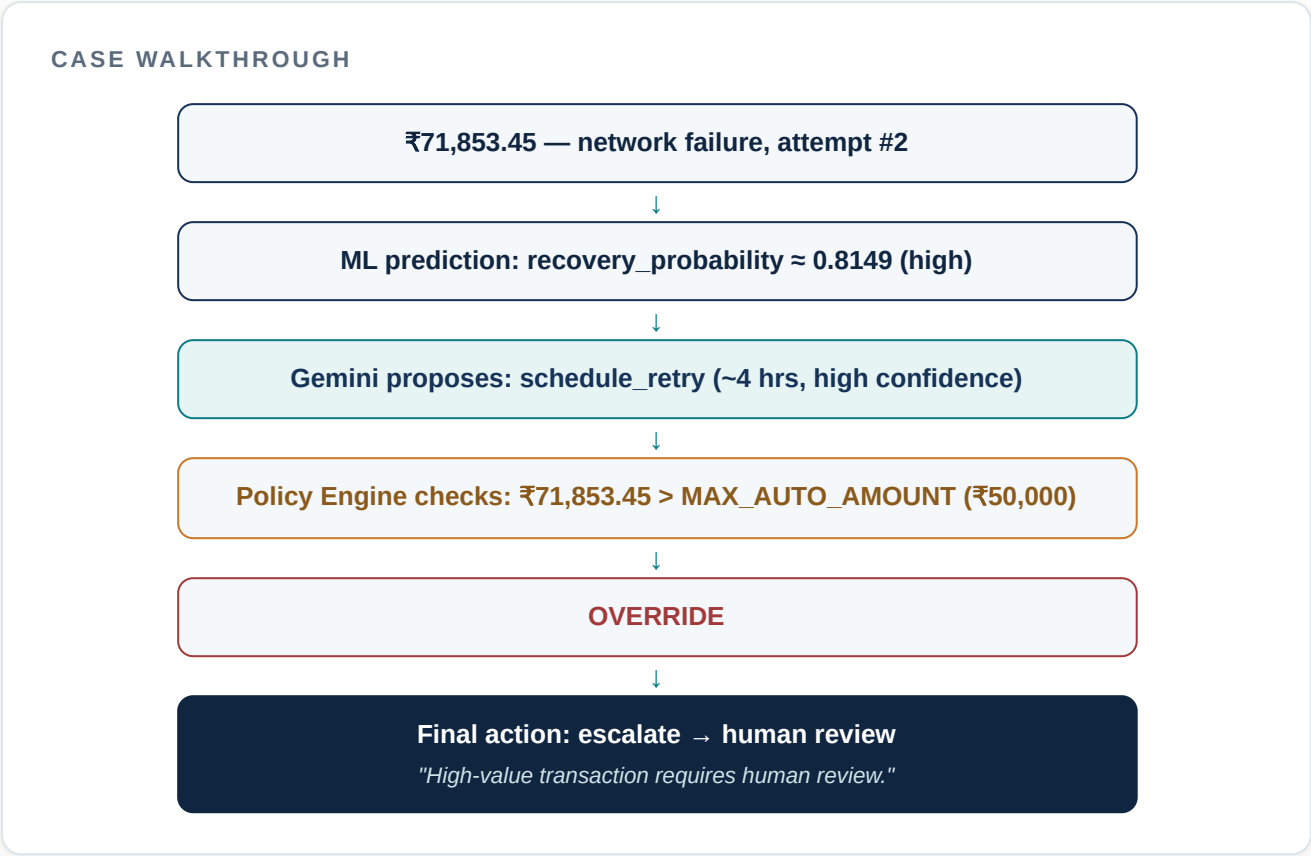
INFRASTRUCTURE / TESTING

- Razorpay Test Mode
- Cloudflare Quick Tunnel (webhook exposure during local dev/demo)

24 Demo Case 1 — Policy Override on a High-Value Transaction

Case ID	PAY219999
Amount	₹71,853.45
Failure reason	network_error
Attempt number	2
Recovery probability	≈ 0.8149
Customer profile	strong historical payment record

Purpose: demonstrate AI reasoning *combined with* deterministic policy governance.



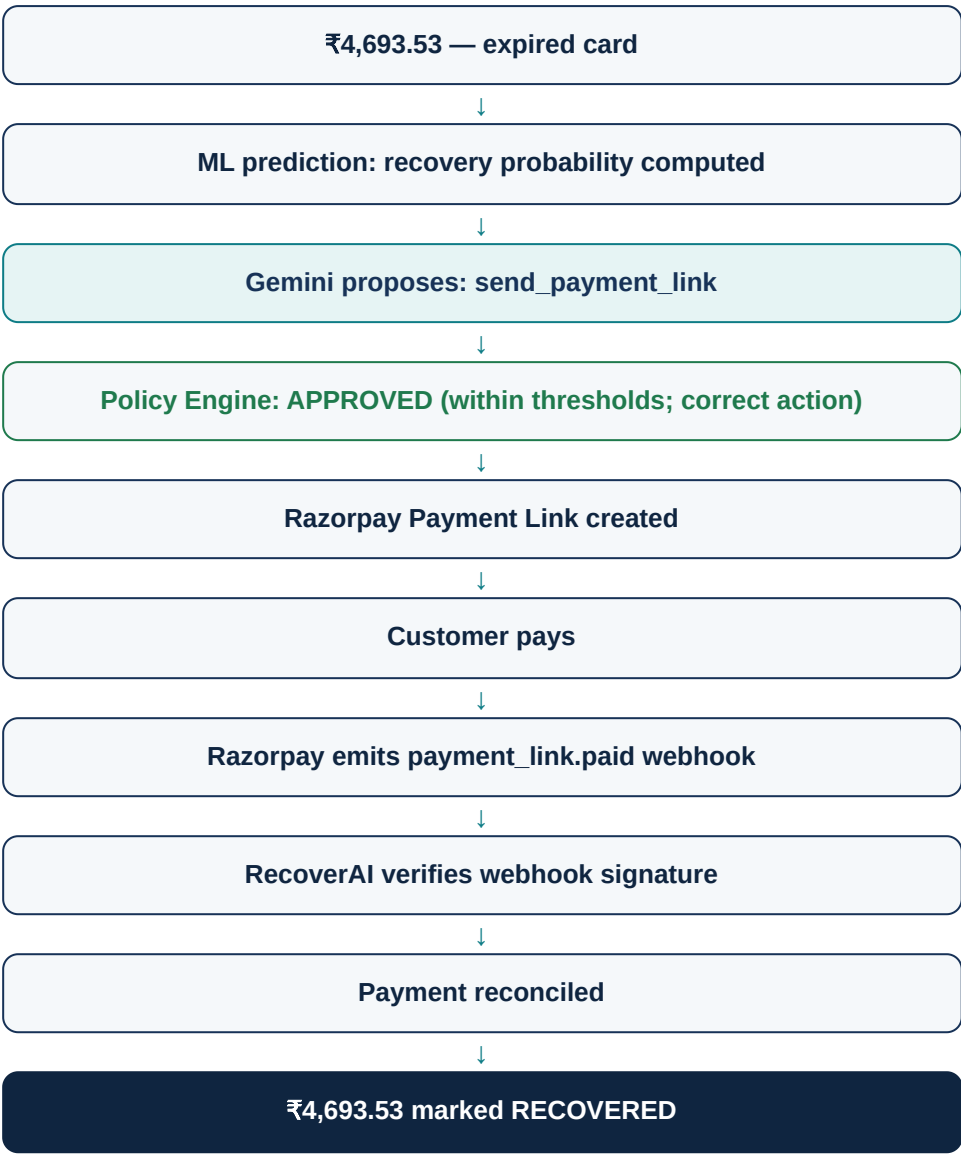
This case demonstrates that a *reasonable* AI proposal can still be *unauthorized*. The AI is not corrected because it was wrong — it is overridden because the merchant's policy imposes a stronger constraint than the AI's own confidence would otherwise justify.

25 Demo Case 2 — Verified End-to-End Recovery

Case ID	PAY200018
Amount	₹4,693.53
Failure reason	expired_card

Purpose: demonstrate actual, verified revenue recovery — not just a recommendation.

CASE WALKTHROUGH



Internal Razorpay identifiers exist for this case but are withheld from external documentation wherever they could expose credentials or sensitive information.

"RecoverAI didn't just recommend a recovery action. It executed a real Razorpay recovery workflow and verified the resulting payment."

26 Batch Experiment & Metrics

To properly evaluate RecoverAI's business value, a controlled batch run across a representative set of failed payments should report:

Metric	Status
Total failed-payment value processed	To be measured
Number of cases processed	To be measured
Number auto-recovered (policy-approved, AI-driven)	To be measured
Total amount verified recovered	To be measured
Recovery rate (verified recovered / total failed value)	To be measured
Percentage escalated to human review	To be measured
Percentage stopped	To be measured
Number of policy overrides of AI proposals	To be measured
Agent source split (LLM-driven vs. fallback-driven decisions)	To be measured

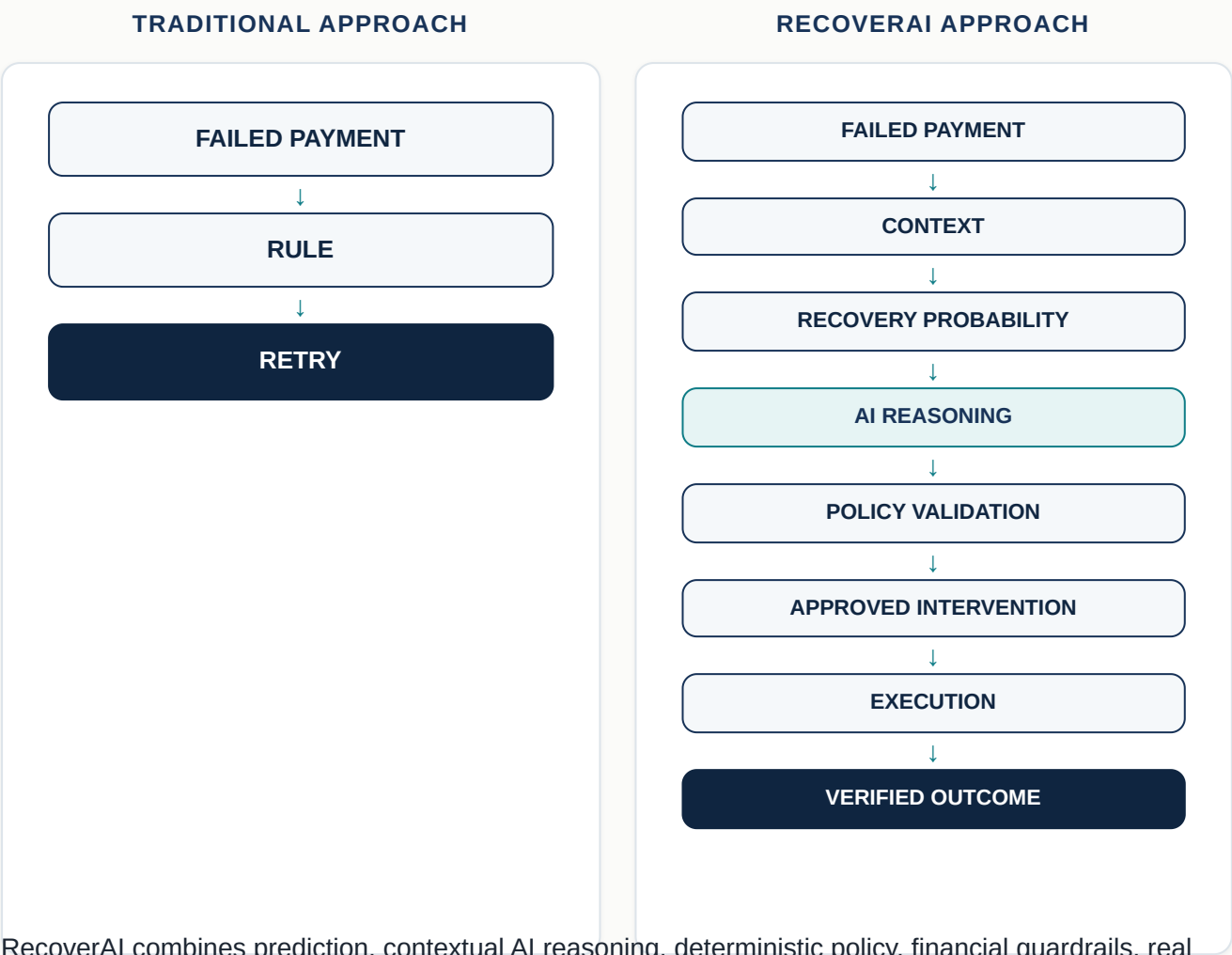
No batch metrics are fabricated in this document. Where a metric has not yet been produced by an actual run, it is explicitly marked **"To be measured"** rather than estimated or invented. The two demo cases in Sections 24–25 are real, individually verified examples; they are not a substitute for aggregate batch results.

27 Key Design Principles

1. **AI proposes. Policy authorizes. Tools execute.**
2. Do not confuse a recovery *action* with recovered *revenue*.
3. High-value financial decisions require stronger controls, not more automation.
4. AI failure should degrade gracefully, never block the pipeline.
5. Every important decision should be auditable end-to-end.
6. Recovery should be measured using actual, verified outcomes — not initiated actions.
7. The objective is not maximum automation. The objective is **maximum safe recovery**.

28 What Makes This Different From a Simple Retry Engine

RecoverAI does not claim to have invented retries, and it does not claim Razorpay lacks recovery mechanisms. The difference is in the decisioning layer sitting above those mechanisms.



RecoverAI combines prediction, contextual AI reasoning, deterministic policy, financial guardrails, real execution, webhook verification, and auditability into a single governed pipeline — rather than a single static rule.

29 Limitations

Transparency about current limitations:

- Some customer-intelligence features used in scoring are derived from **synthetic/project case data**, not a fully live Razorpay customer-history API.
- Payment execution and verification have been demonstrated in **Razorpay Test Mode**, not production.
- Webhook exposure during development relies on a **Cloudflare Quick Tunnel**, a development-grade setup rather than production infrastructure.
- Aggregate batch-level recovery metrics have not yet been finalized (see Section 26); current evidence is anchored in two individually verified demo cases rather than a large-scale run.
- The action vocabulary is intentionally narrow (`retry` , `schedule_retry` , `send_payment_link` , `escalate` , `stop`); more nuanced interventions (e.g., partial refunds, alternate payment method suggestions) are not yet implemented.
- The system has not been evaluated for behavior under Gemini rate-limiting or sustained outage — only for individual call failures.

30 Future Improvements

- Run and publish a full controlled batch experiment across a larger, more diverse set of failed payments to populate the metrics in Section 26.
- Integrate live Razorpay customer-history data to replace synthetic customer-context fields where feasible.
- Expand the action vocabulary (e.g., alternate payment method suggestions, tiered payment-link expiry policies).
- Add configurable, merchant-specific policy thresholds (currently fixed constants) exposed through the dashboard.
- Move webhook infrastructure from a development tunnel to production-grade hosting.
- Add confidence-based routing (e.g., low-confidence LLM proposals routed to fallback logic even when the LLM call itself succeeds).

31 Buildathon Alignment

The Razorpay AI Revenue Recovery Buildathon objective is to identify revenue at risk, determine the right intervention, execute bounded recovery workflows, demonstrate measurable recovered revenue, handle escalation, implement stopping rules, and provide auditability. RecoverAI maps directly onto these criteria:

Evaluation Criterion	How RecoverAI Addresses It
Revenue recovered	Verified via webhook reconciliation (Demo Case 2)
Recovery rate	To be measured via batch experiment
Quality of intervention decisions	Context-aware Gemini reasoning over payment + customer signals
Policy compliance	Deterministic Policy Engine authorizes every action
Safe escalation	Amount-based and probability-based escalation rules
Stopping behavior	Attempt-count and low-probability stop rules
Real execution	Live Razorpay Payment Links (Test Mode)
Verified outcomes	Razorpay webhook signature verification before marking RECOVERED
Audit trail	Full per-case event chain, including LLM provenance
Graceful failure	Deterministic fallback on LLM error/invalid response

32 Pitch Narrative

Opening

More than 50% of customers can drop off when a payment fails on the first attempt. And payment recovery isn't new — modern payment systems already have retries, payment links, and other recovery mechanisms.

So we didn't ask: "How do we recover a failed payment?"

We asked a different question: "When thousands of payments fail every day, what should we do with each one?"

Because recovery isn't one-size-fits-all.

Then introduce three contrasting cases: a ₹500 network failure, a ₹4,700 expired card, and a ₹71,000 network failure — each requiring a different response.

The recovery mechanisms already exist. The harder problem is choosing the right intervention for each payment — while making sure an AI agent doesn't cross a financial boundary.

Positioning

RecoverAI is not another payment retry system. It is an AI-powered decision and orchestration layer for revenue recovery.

We're not replacing existing payment recovery mechanisms. We're making their use intelligent, controlled, and measurable.

The goal isn't maximum automation. The goal is maximum safe recovery.

Closing

Payment recovery already exists. What we built is the intelligence layer that decides how to use it.

RecoverAI takes every failed payment, understands its context, proposes the right intervention, enforces business policy, executes through Razorpay, and verifies the result.

AI proposes. Policy authorizes. Tools execute.

RecoverAI. Detect. Decide. Recover. Verify.

33 Final Mental Model

THE RECOVERAI MENTAL MODEL



Central thesis:

RecoverAI turns payment recovery from a generic action into a contextual, governed, and measurable decision process.

This document distinguishes actual implementation, design intention, demonstrated experiments, and future work throughout. No recovery metrics, revenue figures, or production claims are asserted beyond what is explicitly documented above.